

Software Requirements Specification (SRS) & System Architecture Document

Project: EduFlow — Enterprise-Grade E-Learning Platform

1. Document Control & Executive Summary

- Project Name:** EduFlow E-Learning Management System
- Framework / Language:** Django 5.x / Python 3.11+
- Primary Target:** Production deployment on Render with PostgreSQL, Redis, and Stripe API
- Target Audience:** Developers, Technical Evaluators, Project Reviewers, and Clients

EduFlow is a full-featured, secure, and scalable e-learning platform built using Django's Model-View-Template (MVT) architecture. The platform enables instructors to publish rich multimedia courses and students to browse, enroll via Stripe payment processing, track lesson progress, verify identities via Email OTP and Google OAuth 2.0, and download cryptographically stamped PDF invoices and receipts.

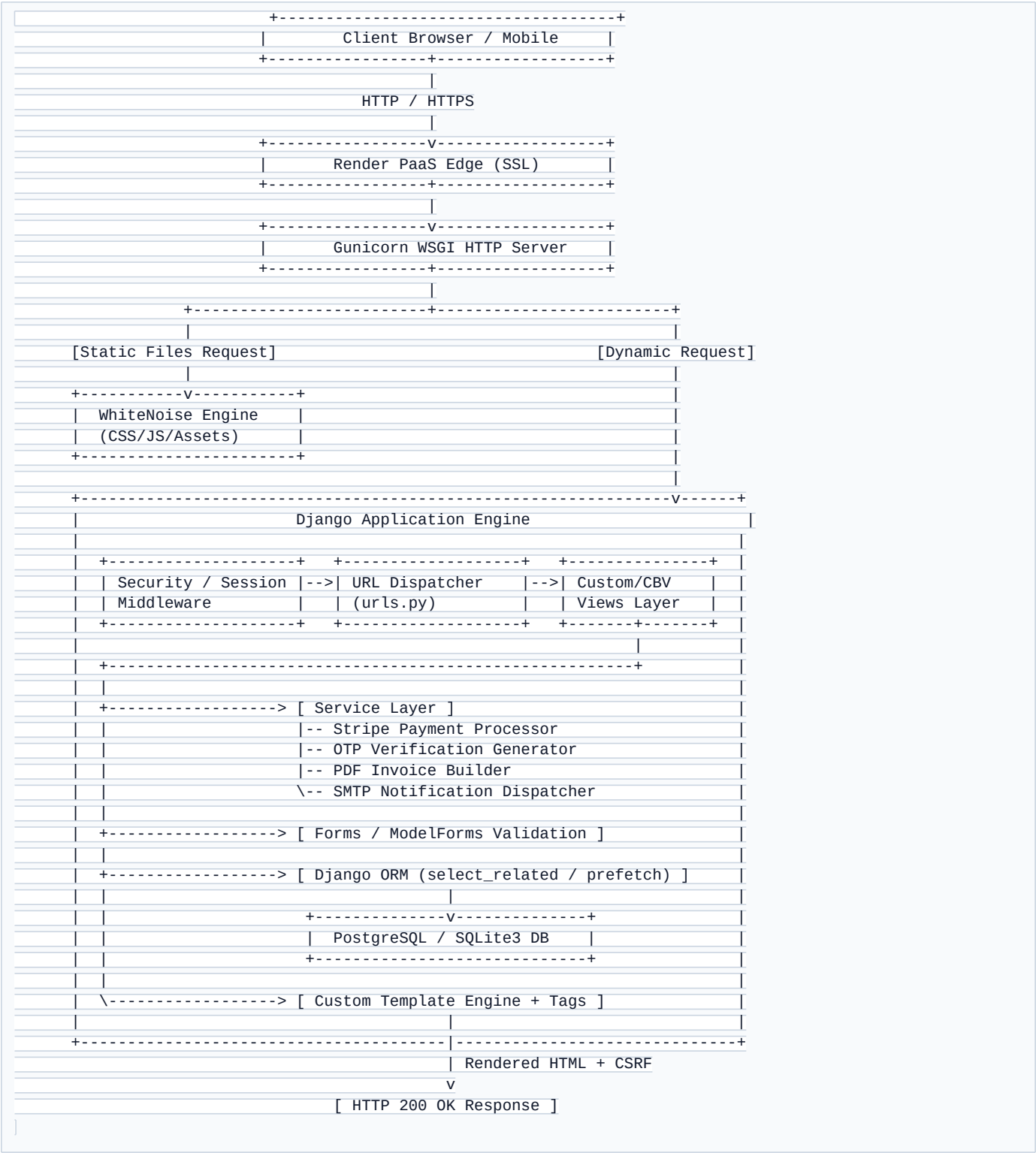
2. Technology Stack & Environment Requirements

Layer	Component / Tool	Version / Standard	Justification / Role
Backend Framework	Django	5.0+	Robust ORM, built-in security (CSRF, XSS, SQLi), enterprise MVT architecture
Language Runtime	Python	3.11 / 3.12	Strongly typed annotations, async I/O compatibility, high performance
Relational Database	PostgreSQL	15+ (Production) / SQLite3 (Dev)	ACID compliance, JSONB support for course curriculum flexibility
Authentication & Auth	Django Auth + <code>django-allauth</code>	Latest	Google OAuth 2.0, session-based auth, email OTP verification
Payment Gateway	Stripe API (<code>stripe-python</code>)	API v2024 / Latest	Hosted checkout sessions, SCA/3D Secure compliance, Webhook lifecycle
PDF & Invoice Engine	<code>xhtml2pdf</code> / <code>ReportLab</code>	Latest	Server-side pixel-perfect PDF receipt and tax invoice generation
Frontend & UI	Django Templates + Tailwind CSS / Bootstrap 5	HTML5, CSS3, Vanilla JS	Lightweight, server-side rendered, mobile-responsive, zero JS framework bloat
Static Assets	WhiteNoise	6.x	High-efficiency static asset caching and compression for PaaS
Deployment & Hosting	Render (Web Service + Managed Postgres)	Production tier	Auto-deploy from GitHub, SSL auto-provisioning, environment isolation

Layer	Component / Tool	Version / Standard	Justification / Role
Email Protocol	SMTP (SendGrid / Gmail App Password)	TLS/SSL	Transactional OTP codes, password resets, registration confirmations

3. System Architecture & High-Level Design

EduFlow follows the clean separation of concerns mandated by Django's **Model-View-Template (MVT)** architecture, augmented by a specialized service layer for third-party external integrations (Stripe, Google OAuth, Email SMTP).



4. Architectural Analysis: OAuth 2.0 vs. JWT

A critical technical decision for this application is choosing between **OAuth 2.0** and **JSON Web Tokens (JWT)**. These concepts are frequently confused by novice developers because they can be used concurrently, but they solve completely distinct problems:

COMPARISON FEATURE	OAUTH 2.0	JSON WEB TOKEN (JWT)
Primary Nature	An Authorization Protocol / Framework (RFC 6749)	A Cryptographic Token Data Format (RFC 7519)
Primary Goal	Delegated access without sharing user credentials to third parties	Compact, stateless data transfer and auth token
Role in EduFlow	Allows users to "Sign in with Google" securely using Google's identity servers.	Optional token format for REST APIs (DRF)
State Management	Leverages Django Session / Database session IDs via HttpOnly secure cookies.	Completely Stateless (Revocation is complex)
Vulnerability Profile	Immune to client-side JS theft if session cookies use `HttpOnly; SameSite=Lax`.	Vulnerable to XSS if in localStorage; CSRF if stored in loose cookies.

Final Architectural Decision for EduFlow:

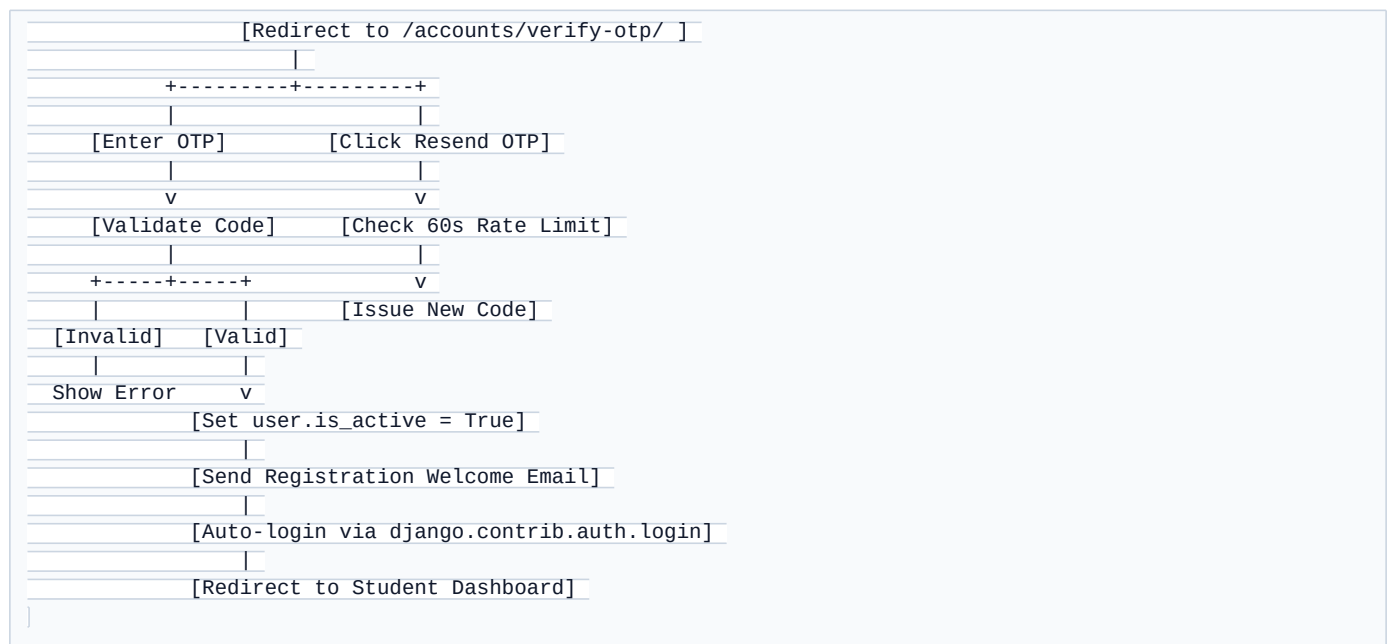
- 1. **Delegated Social Sign-In:** EduFlow implements **OAuth 2.0** (via `django-allauth`) specifically for **Google Sign-In**. EduFlow receives a cryptographically verified token from Google, extracts the user's verified email and name, and issues an internal Django session cookie.
- 2. **Session-Based Authentication:** Because EduFlow is a standard server-rendered Django application (MVT), we use **Django's native cryptographically signed Session Authentication** rather than standalone JWTs. This eliminates token leakage risks through client-side JavaScript (localStorage XSS vulnerabilities) and provides instant session revocation upon logout or password change.

5. Detailed Functional Requirements & Workflows

5.1 User Authentication & Registration with Email OTP

To eliminate disposable bots and unverified accounts, traditional immediate signup is rejected in favor of an **atomic Two-Phase OTP Verification Workflow**:





5.2 Google OAuth 2.0 Authentication Workflow

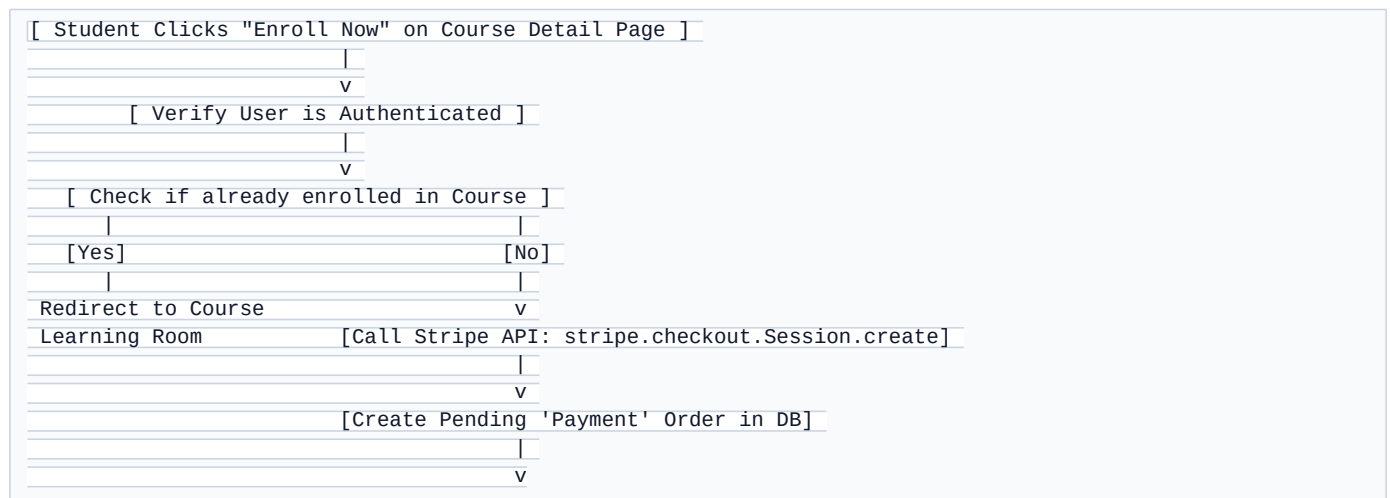
1. User clicks **"Continue with Google"** button on `/accounts/login/` or `/accounts/signup/`.
2. Request redirects to `https://accounts.google.com/o/oauth2/v2/auth` with Client ID, requested scopes (`openid`, `email`, `profile`), and CSRF state token.
3. User consents on Google's authentication page.
4. Google redirects back to EduFlow's callback endpoint: `/accounts/google/login/callback/`.
5. Django validates the state token, exchanges the authorization code for an ID Token via Google's token endpoint.
6. The user profile is retrieved. If a user with that email exists, the Google identity is linked. If not, a new active `User` and `UserProfile` are provisioned immediately (email already verified by Google).
7. An authenticated Django session is created.

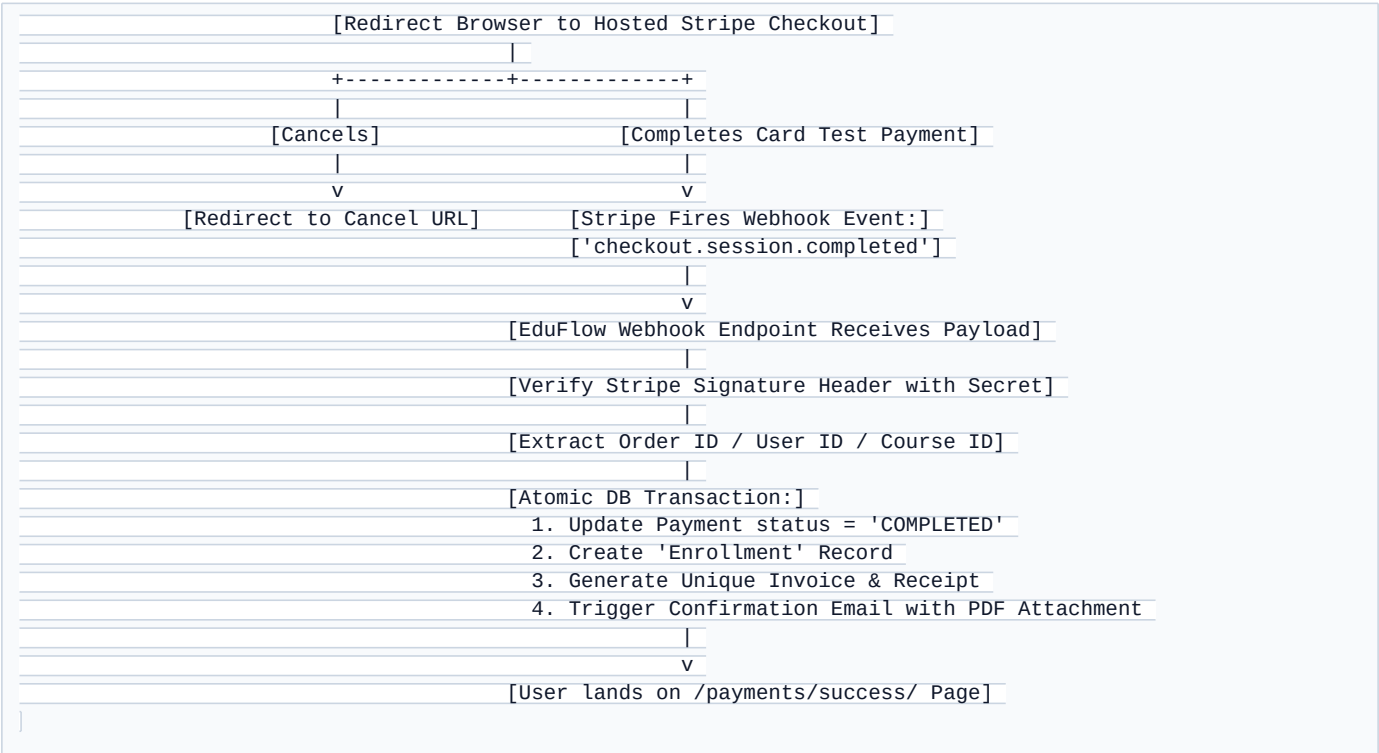
5.3 Course Catalog & Enrollment

- **Course Browsing:** Publicly viewable catalog with real-time category filtering, difficulty rating (Beginner, Intermediate, Advanced), keyword search, and pagination.
- **Curriculum Exploration:** Potential students can preview course outlines, module lists, lesson titles, and free preview videos.
- **Enrollment Check:** When a user accesses a paid lesson, a custom decorator `@enrolled_required` evaluates whether an active `Enrollment` record exists. If not, the user is redirected to the course checkout page.

5.4 Stripe Payment Gateway (Sandbox & Live)

The payment processing flow uses **Stripe Checkout Sessions** and **Stripe Webhooks** for guaranteed reliability:





5.5 Invoicing & Receipt Generation

- **Receipt vs. Invoice:**
- **Receipt:** A lightweight transaction summary indicating payment method, Stripe transaction reference, date, course name, and zero balance due.
- **Tax Invoice:** A formal, numbered legal document (e.g., INV-2026-00481) containing buyer information, seller registration details, tax breakdown, and digital verification seal.
- **Delivery:** Downloadable directly from the student's dashboard in PDF format and delivered automatically as an email attachment upon checkout completion.

6. Django URLs & Routing Architecture

Routes are organized hierarchically across dedicated Django applications:

```
eduflow_project/
|-- core/           (Landing page, Search, Contact, Static pages)
|-- accounts/       (Custom auth, OTP, Profile, Google OAuth)
|-- courses/        (Catalog, Modules, Lessons, Progress tracking)
\-- payments/       (Stripe Checkout, Webhooks, Invoices, Receipts)
```

Complete URL Scheme Table

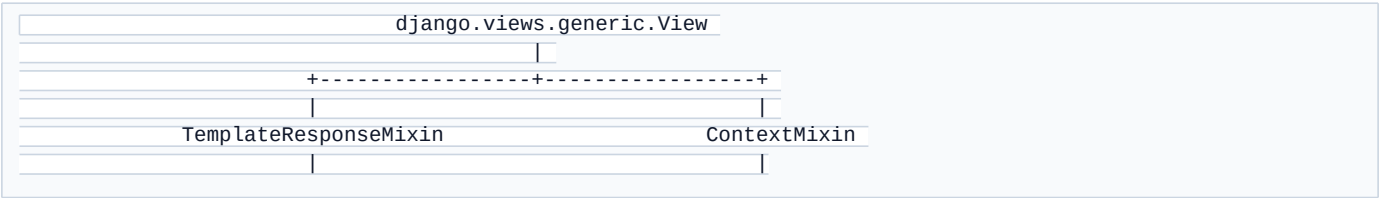
Route Pattern	View Name / Type	Purpose	Auth Required
/	core:home (TemplateView)	Landing page featuring top courses, hero banner, metrics	No
/about/	core:about (TemplateView)	Platform overview, instructor bio, trust badges	No
/accounts/signup/	accounts:SignUpView (FormView)	Registration form; initiates Email OTP	No
/accounts/verify-otp/	accounts:VerifyOTPView (FormView)	OTP submission and account activation	No

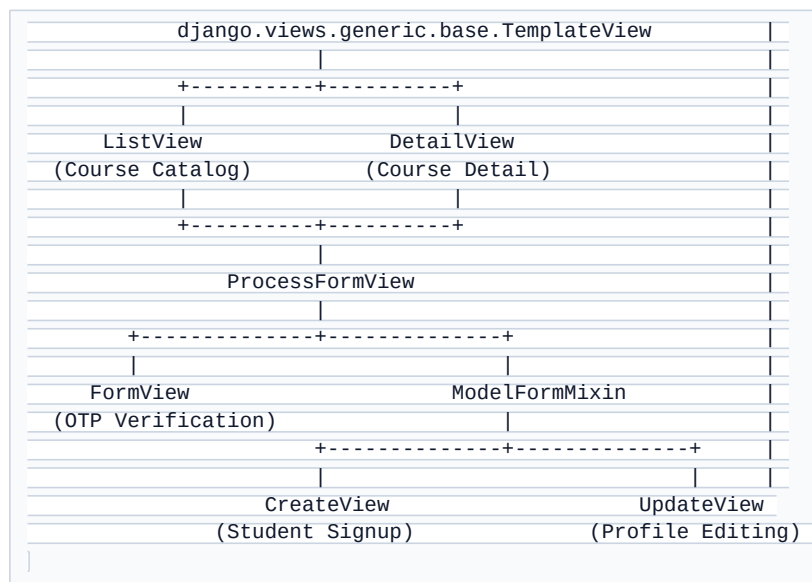
Route Pattern	View Name / Type	Purpose	Auth Required
/accounts/resend-otp/	accounts:ResendOTPView (View)	Throttled trigger to send fresh OTP	No
/accounts/login/	accounts:UserLoginView (LoginView)	Standard credentials login	No
/accounts/logout/	accounts:UserLogoutView (LogoutView)	Logout session and redirects to home	Yes
/accounts/profile/	accounts:ProfileView (DetailView)	Private student portfolio	Yes
/accounts/profile/edit/	accounts:ProfileUpdateView (UpdateView)	Update profile, preferences editing	Yes
/accounts/password-reset/	PasswordResetView (CBV)	Initiates tokenized password reset email	No
/accounts/password-reset/confirm/	PasswordResetConfirmView	Confirms token & sets new password	No
/courses/	courses:CourseListView (ListView)	Searchable, filterable, paginated course catalog	No
/courses/<slug:slug>/	courses:CourseDetailView (DetailView)	Course details, instructor, price, preview	No
/courses/<slug:slug>/learn/	courses:LessonView (DetailView)	Interactive video player & lesson material	Yes (Enrolled)
/courses/lesson/<int:lesson_id>/	courses:MarkCompleteView (APIView)	API endpoint updating progress percentage	Yes (Enrolled)
/payments/checkout/<slug:slug>/	payments:CreateCheckoutSessionView	Creates Stripe Session & redirects	Yes
/payments/success/	payments:PaymentSuccessView	Thank you page after card approval	Yes
/payments/cancel/	payments:PaymentCancelView	Handles aborted checkout attempts	Yes
/payments/webhook/stripe/	payments:StripeWebhookView (APIView)	Stripe Connect server-to-server transaction listener	No (Stripe Sig)
/payments/invoice/<str:invoice_id>/	payments:DownloadInvoicePDFView	Generates & streams PDF invoice	Yes (Owner)
/payments/receipt/<str:transaction_id>/	payments:DownloadReceiptPDFView	Generates & streams PDF receipt	Yes (Owner)

7. Advanced Django Views & Class-Based View (CBV) Design

Class-Based Views (CBVs) enforce modularity, DRY (Don't Repeat Yourself) design, and reusable behavioral mixins.

7.1 View Class Hierarchy Diagram





7.2 Custom Access Control Mixins

```

# accounts/mixins.py
from django.contrib.auth.mixins import AccessMixin
from django.shortcuts import redirect
from django.urls import reverse_lazy
from django.contrib import messages

class EnrolledCourseRequiredMixin(AccessMixin):
    """
    CBV Mixin to verify that the logged-in student has an active enrollment
    record for the requested course before allowing lesson access.
    """
    def dispatch(self, request, *args, **kwargs):
        if not request.user.is_authenticated:
            return self.handle_no_permission()

        course_slug = self.kwargs.get("slug")
        has_access = request.user.enrollments.filter(
            course__slug=course_slug,
            is_active=True
        ).exists()

        if not has_access:
            messages.warning(request, "You must enroll in this course to access the classroom lessons.")
            return redirect("courses:course_detail", slug=course_slug)

        return super().dispatch(request, *args, **kwargs)
  
```

8. Database Architecture & Django Models

8.1 Entity Relationship Diagram (Mermaid)

```

erDiagram
    USER ||--o| USER_PROFILE : "has"
    USER ||--o{ EMAIL_OTP : "issues"
    USER ||--o{ ENROLLMENT : "purchases"
    USER ||--o{ PAYMENT_TRANSACTION : "executes"
    USER ||--o{ LESSON_PROGRESS : "completes"
    COURSE_CATEGORY ||--o{ COURSE : "categorizes"
    USER ||--o{ COURSE : "instructs"
    COURSE ||--o{ COURSE_MODULE : "contains"
    COURSE_MODULE ||--o{ LESSON : "contains"
  
```

```

COURSE ||--o{ ENROLLMENT : "enrolled_in"
COURSE ||--o{ PAYMENT_TRANSACTION : "billed_for"
-
LESSON ||--o{ LESSON_PROGRESS : "tracked_by"
PAYMENT_TRANSACTION ||--o{ INVOICE : "generates"
-
USER {
  int id PK
  string username
  string email UK
  string password
  boolean is_active
  boolean is_staff
  datetime date_joined
}
-
USER_PROFILE {
  int id PK
  int user_id FK
  string avatar
  text bio
  string headline
  string phone_number
  string preferred_currency
}
-
EMAIL_OTP {
  int id PK
  int user_id FK
  string otp_code
  datetime created_at
  datetime expires_at
  boolean is_verified
  int attempts_count
}
-
COURSE {
  int id PK
  string title
  string slug UK
  text short_description
  text full_description
  decimal price
  string level
  int category_id FK
  int instructor_id FK
  boolean is_published
  datetime created_at
}
-
COURSE_MODULE {
  int id PK
  int course_id FK
  string title
  int order_number
}
-
LESSON {
  int id PK
  int module_id FK
  string title
  string video_url
  text content
  int duration_seconds
  int order_number
  boolean is_preview
}
-

```



```

    ENROLLMENT {
        int id PK
        int user_id FK
        int course_id FK
        datetime enrolled_at
        boolean is_active
        decimal progress_percent
    }

    PAYMENT_TRANSACTION {
        int id PK
        int user_id FK
        int course_id FK
        string stripe_checkout_session_id UK
        string stripe_payment_intent_id
        decimal amount
        string currency
        string status
        datetime created_at
    }

    INVOICE {
        int id PK
        int transaction_id FK
        string invoice_number UK
        string billing_name
        string billing_email
        decimal subtotal
        decimal tax_amount
        decimal total_amount
        datetime issued_at
        string pdf_file
    }
}

```

8.2 Django Models Specifications with Properties & Indexes

```

# courses/models.py
from django.db import models
from django.conf import settings
from django.utils.text import slugify

class Course(models.Model):
    LEVEL_CHOICES = (
        ("BEGINNER", "Beginner"),
        ("INTERMEDIATE", "Intermediate"),
        ("ADVANCED", "Advanced"),
    )

    title = models.CharField(max_length=200, db_index=True)
    slug = models.SlugField(max_length=220, unique=True, db_index=True)
    instructor = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="authored_courses"
    )
    category = models.ForeignKey(
        "CourseCategory",
        on_delete=models.SET_NULL,
        null=True,
        related_name="courses"
    )
    price = models.DecimalField(max_digits=8, decimal_places=2)
    level = models.CharField(max_length=15, choices=LEVEL_CHOICES, default="BEGINNER")
    is_published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

```

```

class Meta:
    ordering = ["-created_at"]
    indexes = [
        models.Index(fields=["slug", "is_published"]),
        models.Index(fields=["price"]),
    ]

    def __str__(self):
        return self.title

    @property
    def total_lessons_count(self) -> int:
        """Calculates total lessons across all modules efficiently."""
        return Lesson.objects.filter(module__course=self).count()

    @property
    def is_free(self) -> bool:
        """Calculated model property."""
        return self.price == 0.00

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.title)
        super().save(*args, **kwargs)

```

8.3 ORM Optimization: filter() vs get(), select_related vs prefetch_related

Understanding Django's ORM nuances is mandatory for high-performance applications:

- **get() vs filter():**
 - `Course.objects.get(slug=slug)`: Queries for exactly **one** object. Raises `Course.DoesNotExist` if no records match, or `Course.MultipleObjectsReturned` if more than one matches. Use this when retrieving by guaranteed unique constraints (`id`, `slug`).
 - `Course.objects.filter(category=cat)`: Returns a lazy `QuerySet` (0, 1, or many objects). Never raises `DoesNotExist`. Use `filter(...).first()` if you want the first match gracefully returning `None` if missing.
- **The N+1 Query Problem & Solutions:**
 - **select_related**: Uses an SQL `INNER JOIN` or `LEFT OUTER JOIN`. Used exclusively for **Single-Valued Relationships** (`ForeignKey`, `OneToOneField`). python # Efficient: 1 single SQL JOIN query fetching course and instructor details `courses = Course.objects.select_related('instructor', 'category').filter(is_published=True)`
 - **prefetch_related**: Performs two separate SQL queries and combines them in Python memory. Used for **Multi-Valued Relationships** (`ManyToManyField`, `Reverse ForeignKey`). python # Efficient: 2 queries instead of 1 + N queries for modules & lessons `course = Course.objects.prefetch_related('modules__lessons').get(slug='django-masterclass')`

9. Forms, ModelForms, & Advanced Validation

Django forms handle data cleansing, CSRF security, widget styling, and cross-field validation.

```

# accounts/forms.py
from django import forms
from django.contrib.auth.models import User
from django.core.exceptions import ValidationError
import re

class StudentRegistrationForm(forms.ModelForm):
    password = forms.CharField(
        widget=forms.PasswordInput(attrs={
            "class": "form-control rounded-lg border-slate-300",
            "placeholder": "Minimum 8 characters with digits & symbols"
        })
    )

```

```

    )
    confirm_password = forms.CharField(
        widget=forms.PasswordInput(attrs={
            "class": "form-control rounded-lg border-slate-300",
            "placeholder": "Repeat password"
        })
    )
)

class Meta:
    model = User
    fields = ("first_name", "last_name", "username", "email")
    widgets = {
        "first_name": forms.TextInput(attrs={"class": "form-control"}),
        "last_name": forms.TextInput(attrs={"class": "form-control"}),
        "username": forms.TextInput(attrs={"class": "form-control"}),
        "email": forms.EmailInput(attrs={"class": "form-control"}),
    }

def clean_email(self):
    """Custom field validator: Email uniqueness & domain checking."""
    email = self.cleaned_data.get("email", "").lower().strip()
    if User.objects.filter(email=email).exists():
        raise ValidationError("An account with this email address already exists.")
    return email

def clean(self):
    """Cross-field validation: Password match & strength."""
    cleaned_data = super().clean()
    p1 = cleaned_data.get("password")
    p2 = cleaned_data.get("confirm_password")

    if p1 and p2 and p1 != p2:
        self.add_error("confirm_password", "Passwords do not match.")

    if p1 and len(p1) < 8:
        self.add_error("password", "Password must be at least 8 characters long.")

    return cleaned_data

```

10. Exception Handling, Custom Exceptions & Decorators

10.1 Custom Exception Classes

```

# core/exceptions.py

class EduFlowBaseException(Exception):
    """Base exception for all domain-specific platform errors."""
    def __init__(self, message="An internal business logic error occurred.", code="INTERNAL_ERROR"):
        self.message = message
        self.code = code
        super().__init__(self.message)

class PaymentVerificationFailedException(EduFlowBaseException):
    """Raised when Stripe webhook signature or charge verification fails."""
    def __init__(self, message="The payment verification check failed."):
        super().__init__(message=message, code="PAYMENT_VERIFICATION_FAILED")

class CourseAccessDeniedException(EduFlowBaseException):
    """Raised when a student attempts unauthorized lesson access."""
    def __init__(self, message="Enrollment required to access course content."):
        super().__init__(message=message, code="ACCESS_DENIED")

class OTPExpiredException(EduFlowBaseException):
    """Raised when an OTP submission exceeds the permitted validity window."""
    def __init__(self, message="This OTP has expired. Please request a new verification code."):

```

```
super().__init__(message=message, code="OTP_EXPIRED")
```

10.2 Custom Function-Based Decorators

```
# courses/decorators.py
from functools import wraps
from django.shortcuts import get_object_or_404, redirect
from django.contrib import messages
from .models import Course

def enrolled_required(view_func):
    """
    Decorator for views that checks that the user is logged in and enrolled
    in the course corresponding to the 'slug' parameter in the URL.
    """
    @wraps(view_func)
    def _wrapped_view(request, slug, *args, **kwargs):
        if not request.user.is_authenticated:
            messages.info(request, "Please log in to access this lesson.")
            return redirect(f"/accounts/login/?next={request.path}")

        course = get_object_or_404(Course, slug=slug)
        is_enrolled = request.user.enrollments.filter(course=course, is_active=True).exists()

        if not is_enrolled and not request.user.is_staff:
            messages.error(request, "You are not enrolled in this course.")
            return redirect("courses:course_detail", slug=slug)

        return view_func(request, slug, *args, **kwargs)
    return _wrapped_view
```

11. Django Templating Engine, Inheritance & Custom Template Tags

11.1 Template Architecture

Templates use strict hierarchical inheritance to maintain DRY HTML structures:

```
templates/
|-- base.html                (Master skeleton: HTML5 head, navbar, alerts, footer, scripts)
|-- includes/
|   |-- _navbar.html         (Dynamic auth state buttons, cart icon, avatar)
|   |-- _footer.html         (Legal links, copyright, theme switcher)
|   \-- _messages.html       (Django alert messages formatted with Tailwind/Bootstrap)
|-- accounts/
|   |-- signup.html          (Extends base.html)
|   |-- verify_otp.html       (OTP digit entry cards)
|   |-- login.html           (Credentials form + Google OAuth button)
|   \-- profile.html         (Student profile card & enrollment history)
|-- courses/
|   |-- course_list.html      (Card grid, search form, paginator)
|   |-- course_detail.html    (Course trailer, syllabus accordion, enrollment CTA)
|   \-- lesson_player.html    (Video stream container, lesson checklist sidebar)
\-- payments/
    |-- invoice_template.html (Clean printable layout rendered to PDF)
    \-- receipt_template.html (Transaction receipt rendered to PDF)
```

11.2 Custom Template Tags and Filters

Custom filters and simple tags extend Django's template rendering capabilities:

```
# courses/templatetags/course_extras.py
from django import template
from django.utils.safestring import mark_safe
import math
```

```

-
register = template.Library()
-
@register.filter(name="format_duration")
def format_duration(seconds: int) -> str:
    """Converts raw seconds into a human-readable duration (e.g. 1h 45m)."""
    if not seconds:
        return "0m"
    hours = seconds // 3600
    minutes = (seconds % 3600) // 60
    if hours > 0:
        return f"{hours}h {minutes}m"
    return f"{minutes}m"
-
@register.simple_tag
def calculate_progress_badge(completed_count: int, total_count: int) -> str:
    """Calculates completion percentage and returns a colored HTML badge."""
    if total_count == 0:
        pct = 0
    else:
        pct = int(math.floor((completed_count / total_count) * 100))
    -
    if pct == 100:
        color_class = "bg-green-100 text-green-800"
    elif pct > 50:
        color_class = "bg-blue-100 text-blue-800"
    else:
        color_class = "bg-yellow-100 text-yellow-800"
    -
    html = f'<span class="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium {color_class}">{pct}%</span>'
    return mark_safe(html)
-

```

Usage inside `templates/courses/lesson_player.html`:

```

{% load course_extras %}

<div class="lesson-meta">
  <span>Duration: {{ lesson.duration_seconds|format_duration }}</span>
  {% calculate_progress_badge completed_lessons_count total_lessons_count %}
</div>

```

12. Complete Stripe Sandbox & Webhook Integration

12.1 Checkout Session Generator View

```

# payments/views.py
import stripe
from django.conf import settings
from django.views import View
from django.shortcuts import get_object_or_404, redirect
from django.http import JsonResponse
from courses.models import Course
from .models import PaymentTransaction
-
stripe.api_key = settings.STRIPE_SECRET_KEY
-
class CreateCheckoutSessionView(View):
    def post(self, request, course_slug):
        course = get_object_or_404(Course, slug=course_slug, is_published=True)
        -
        # Prevent duplicate payment
        if request.user.enrollments.filter(course=course, is_active=True).exists():
            return redirect("courses:course_detail", slug=course.slug)
        -

```

```

        checkout_session = stripe.checkout.Session.create(
            customer_email=request.user.email,
            payment_method_types=["card"],
            line_items=[
                {
                    "price_data": {
                        "currency": "usd",
                        "product_data": {
                            "name": course.title,
                            "description": course.short_description[:200],
                        },
                        "unit_amount": int(course.price * 100), # In cents
                    },
                    "quantity": 1,
                }
            ],
            mode="payment",
            metadata={
                "user_id": str(request.user.id),
                "course_id": str(course.id)
            },
            success_url=request.build_absolute_uri("/payments/success/?session_id={CHECKOUT_SESSION_ID}"),
            cancel_url=request.build_absolute_uri(f"/courses/{course.slug}/"),
        )

        PaymentTransaction.objects.create(
            user=request.user,
            course=course,
            stripe_checkout_session_id=checkout_session.id,
            amount=course.price,
            currency="usd",
            status="PENDING"
        )

        return redirect(checkout_session.url, code=303)
    )

```

12.2 Production Stripe Webhook Receiver

The webhook ensures payments are fulfilled even if the user closes their browser before reaching the return URL:

```

# payments/webhooks.py
import stripe
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST
from django.conf import settings
from django.contrib.auth.models import User
from courses.models import Course, Enrollment
from .models import PaymentTransaction, Invoice
from .services import generate_invoice_pdf, dispatch_order_confirmation_email

@csrf_exempt
@require_POST
def stripe_webhook(request):
    payload = request.body
    sig_header = request.META.get("HTTP_STRIPE_SIGNATURE")

    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, settings.STRIPE_WEBHOOK_SECRET
        )
    except (ValueError, stripe.error.SignatureVerificationError):
        return HttpResponse(status=400)

    if event["type"] == "checkout.session.completed":
        session = event["data"]["object"]
        metadata = session.get("metadata", {})

```

```

user_id = metadata.get("user_id")
course_id = metadata.get("course_id")

if user_id and course_id:
    user = User.objects.get(id=int(user_id))
    course = Course.objects.get(id=int(course_id))

    # 1. Update Payment Transaction
    transaction = PaymentTransaction.objects.get(
        stripe_checkout_session_id=session["id"]
    )
    transaction.status = "COMPLETED"
    transaction.stripe_payment_intent_id = session.get("payment_intent")
    transaction.save()

    # 2. Grant Enrollment
    Enrollment.objects.get_or_create(
        user=user,
        course=course,
        defaults={"is_active": True}
    )

    # 3. Create Formal Invoice Record & Generate PDF
    invoice = Invoice.objects.create(
        transaction=transaction,
        billing_name=user.get_full_name() or user.username,
        billing_email=user.email,
        subtotal=transaction.amount,
        tax_amount=0.00,
        total_amount=transaction.amount,
    )
    pdf_path = generate_invoice_pdf(invoice)

    # 4. Dispatch Email with Attachment
    dispatch_order_confirmation_email(user, course, invoice, pdf_path)

return HttpResponse(status=200)

```

13. PDF Invoice & Receipt Generation Architecture

PDF generation converts clean, stylized HTML/CSS directly into printable PDFs via `xhtml2pdf` :

```

# payments/services.py
from io import BytesIO
from django.template.loader import get_template
from xhtml2pdf import pisa
from django.core.files.base import ContentFile
import uuid

def generate_invoice_pdf(invoice) -> bytes:
    """
    Renders an HTML invoice template and converts it into a PDF byte stream.
    """
    template = get_template("payments/invoice_template.html")
    context = {
        "invoice": invoice,
        "transaction": invoice.transaction,
        "course": invoice.transaction.course,
        "company_name": "EduFlow Learning Inc.",
        "company_address": "100 Innovation Way, Suite 400, San Francisco, CA",
        "tax_id": "US-948201948",
    }
    html = template.render(context)
    result = BytesIO()
    pdf = pisa.pisaDocument(BytesIO(html.encode("UTF-8")), result)

```

```
--
if not pdf.err:
    pdf_bytes = result.getvalue()
    # Save to Model FileField
    invoice.pdf_file.save(f"invoice_{invoice.invoice_number}.pdf", ContentFile(pdf_bytes))
    return pdf_bytes
return None
|
```

14. Email Subsystem & Transactional Notifications

All outgoing emails use modular HTML templates with plain-text fallback options:

1. **Registration OTP Email:** Contains dynamic 6-digit verification code with a bold 10-minute expiry warning.
2. **Registration Confirmation Email:** Welcomes the activated user, links to the course catalog, and introduces platform features.
3. **Password Reset Email:** Sends cryptographically timed token links (`django.contrib.auth.tokens.default_token_generator`).
4. **Password Reset Confirmation Email:** Security notice dispatched when a password change occurs, notifying the user to contact support immediately if unprompted.
5. **Payment Order Confirmation & Receipt Email:** Includes course access details and attaches the generated PDF invoice.

15. Security, Session & Exception Defenses

- **CSRF Protection:** Every state-altering `POST` form includes `{% csrf_token %}`. Stripe Webhooks are exempted via `@csrf_exempt` but secured via cryptographic HMAC-SHA256 signature verification.
- **SQL Injection Prevention:** 100% of queries use the Django ORM parameterized interface; zero raw string SQL interpolations.
- **Brute-Force OTP Throttling:** `EMAIL_OTP` checks enforce an `attempts_count` ceiling (maximum 5 attempts). Once breached, the token is invalidated, requiring a fresh code generation.
- **Cross-Site Scripting (XSS):** Django's auto-escaping is active across all template contexts. Unescaped HTML (`mark_safe`) is restricted exclusively to trusted internal tags.
- **Secure Cookies:** Production environments enforce:
 - `SESSION_COOKIE_HTTPONLY = True`
 - `SESSION_COOKIE_SECURE = True`
 - `CSRF_COOKIE_SECURE = True`
 - `SECURE_BROWSER_XSS_FILTER = True`

16. Complete Deployment Architecture on Render

16.1 Directory & Config Structure

```
eduflow/
|-- build.sh                (Bash script for build hooks)
|-- render.yaml            (Render Infrastructure-as-Code blueprint)
|-- requirements.txt        (Pinned Python dependencies)
|-- manage.py
|-- eduflow_project/
|   |-- settings.py
|   |-- urls.py
|   \-- wsgi.py
|
```

16.2 Build Script (`build.sh`)


```
#!/usr/bin/env bash
# Exit on error
set -o errexit

echo "Installing project dependencies..."
pip install --upgrade pip
pip install -r requirements.txt

echo "Collecting static assets..."
python manage.py collectstatic --no-input

echo "Executing database migrations..."
python manage.py migrate --no-input
```

16.3 Render Blueprint File (render.yaml)

```
services:
- type: web
  name: eduflow-app
  env: python
  region: oregon
  plan: free
  buildCommand: "./build.sh"
  startCommand: "gunicorn eduflow_project.wsgi:application --workers 2 --bind 0.0.0.0:$PORT"
  envVars:
    - key: PYTHON_VERSION
      value: 3.11.8
    - key: DEBUG
      value: "False"
    - key: SECRET_KEY
      generateValue: true
    - key: DATABASE_URL
      fromDatabase:
        name: eduflow-postgres
        property: connectionString
    - key: STRIPE_PUBLIC_KEY
      sync: false
    - key: STRIPE_SECRET_KEY
      sync: false
    - key: STRIPE_WEBHOOK_SECRET
      sync: false
    - key: EMAIL_HOST_USER
      sync: false
    - key: EMAIL_HOST_PASSWORD
      sync: false

databases:
- name: eduflow-postgres
  plan: free
  region: oregon
```

17. Project File & Directory Blueprint

```
eduflow/
|-- accounts/
|   |-- migrations/
|   |-- forms.py           # UserCreationForm, OTPForm, ProfileEditForm
|   |-- models.py          # UserProfile, EmailOTP
|   |-- views.py           # SignUpView, VerifyOTPView, LoginView
|   |-- urls.py            # Auth routing endpoints
|   \-- signals.py         # Auto-create UserProfile on User creation
|-- courses/
|   |-- migrations/
|   |-- models.py          # Course, Module, Lesson, Enrollment, Progress
```

```

| | | -- views.py # CourseListView, CourseDetailView, LessonView
| | | -- urls.py
| | | -- decorators.py # @enrolled_required
| | | \-- templatetags/
| | | \-- course_extras.py# Custom filters & calculation badges
| -- payments/
| | -- migrations/
| | | -- models.py # PaymentTransaction, Invoice
| | | -- views.py # CreateCheckoutSessionView, PDFDownloadViews
| | | -- webhooks.py # Stripe webhook listener
| | | -- services.py # xhtml2pdf generation, email dispatcher
| | | \-- urls.py
| -- core/
| | | -- exceptions.py # EduFlow domain exception hierarchy
| | | -- views.py # HomeView, ErrorPages (404, 500)
| | | \-- urls.py
| -- static/
| | | -- css/
| | | | -- js/
| | | \-- images/
| -- templates/ # As detailed in Section 11.1
| -- build.sh
| -- render.yaml
| -- requirements.txt
\-- manage.py

```

18. Beginner Implementation Roadmap & Step-by-Step Milestones

- **Milestone 1 (Foundation):** Setup virtual environment, initialize Django project with apps (`accounts` , `courses` , `payments` , `core`), configure `settings.py` with environment variable loading via `python-decouple` or `environ` .
- **Milestone 2 (Authentication Core):** Implement `StudentRegistrationForm` , `EmailOTP` model, SMTP settings with test email logging backend, and OTP verification CBVs.
- **Milestone 3 (Google OAuth 2.0):** Register project in Google Cloud Console, configure OAuth credentials, and connect `django-allauth` .
- **Milestone 4 (Courses & Learning MVT):** Create models for `Course` , `Module` , and `Lesson` . Write and execute initial migrations. Build `CourseListView` and `CourseDetailView` with Tailwind template components.
- **Milestone 5 (Stripe Sandbox Integration):** Setup Stripe Developer account, retrieve Test Secret/Publishable keys, write `CreateCheckoutSessionView` , and configure Stripe CLI to forward sandbox webhooks locally.
- **Milestone 6 (Invoices & Receipts):** Configure `xhtml2pdf` , design invoice HTML templates, verify PDF binary stream downloads.
- **Milestone 7 (Render Production Deployment):** Create `build.sh` and `render.yaml` , push repository to GitHub, link repository in Render dashboard, provision managed PostgreSQL database, and run live verification.

19. Comprehensive Technical Viva / Client Demo Preparation

During technical review and client evaluations, be prepared to answer these core architecture questions:

1. **Why use Django Class-Based Views over Function-Based Views (FBVs)?**
 - *Answer:* CBVs promote object-oriented reusability. Instead of repeating boilerplate request checks and queryset fetching in every view, CBVs (`ListView` , `DetailView` , `CreateView`) encapsulate standard logic into well-tested mixins (`LoginRequiredMixin` , `UserPassesTestMixin`), drastically reducing code duplication and bug surfaces.
2. **How does EduFlow prevent the N+1 database queries problem?**
 - *Answer:* By utilizing `select_related` for single-valued relationships (e.g., loading a course's instructor and category using an SQL JOIN) and `prefetch_related` for multi-valued sets (e.g., prefetching modules and lessons for a course syllabus in a single coordinated query).
3. **Why use Session Authentication rather than storing JWTs in localStorage?**

- *Answer:* Storing JWTs in browser `localStorage` leaves tokens vulnerable to Cross-Site Scripting (XSS) extraction. Django's session IDs are stored inside `HttpOnly`, `SameSite=Lax`, and `Secure` cookies, rendering them inaccessible to malicious client-side JavaScript.

4. How are duplicate course enrollments prevented if a user double-clicks checkout?

- *Answer:* Through three layers of defense: (1) View-level check preventing checkout session creation if an active `Enrollment` already exists; (2) Idempotent Stripe checkout sessions; and (3) Database-level `unique_together = ('user', 'course')` constraint on the `Enrollment` model.

5. How do you test Stripe Webhooks locally without deploying to a public URL?

- *Answer:* By installing the official Stripe CLI tool and running: `stripe listen --forward-to localhost:8000/payments/webhook/stripe/`. The CLI provides a local test webhook signing secret.

6. What occurs if the customer's payment succeeds on Stripe, but their internet drops before returning to EduFlow?

- *Answer:* The platform relies on the asynchronous **Stripe Webhook** (`checkout.session.completed`). Because webhooks run server-to-server with automatic retries from Stripe's infrastructure, the enrollment, invoice generation, and confirmation emails execute regardless of the student's browser state.